Lab Manual

CS226L – Computer Organization and Assembly Language Lab

Lab No: **05**

Topic: **Assembly Language Introduction**

Class: **BSDS**

Semester: **3**

Session: Spring**, 2026**

Lab Instructor: Aamar Khan

Lab Date: 12/10/26

Lab Time: 5:00 - 8:00

Air University Islamabad

FACULTY OF COMPUTING & ARTIFICAL INTELLIGENCE

Department of Creative Technologies

Instructions

Submission: Use proper naming convention for your submission file. Name the submission file as Lab_NO_DEGREE_ROLLNUM (e.g. Lab_01_BSAI_00000). Submit the file on Google Classroom within the deadline. Failure to submit according to the above format would result in deduction of 10% marks. Submissions on the email will not be accepted.

Objectives

In this lab, you will learn:

- About Structure ADT.
- How to define a Structure, initialize and refer to individual members of a Structure.

Objectives

In this lab, you will learn how to:

- Evaluate the if-else condition.
- Control flow of program based on defined condition.
- User static value and user input number.
- Student tasks.

8086 Assembly Program: Complex Formula Evaluation

new open examples save

```
01 org 100h
02
03 .data
04     A DW 20
05     B DW 5
06     C DW 3
07     D DW 4
08     E DW 3
09     T DW ?
10     Y DW ?
11
12 .code
13 main proc
14     MOV AX, @data
15     MOV DS, AX
16
17     MOV AX, A
18     SUB AX, B
19     MOV BX, AX
20
21     MOV AX, D
22     MUL E
23     ADD AX, C
24     MOV T, AX
25
26     MOV AX, BX
27     MOV DX, 0
28     DIV T
29
30     MOV Y, AX
31
32     MOV AH, 4Ch
33     INT 21h
34 main endp
35 end main
```

2. Formula Breakdown

Mathematical Expression: Y = (A - B) / (C - (D * E))

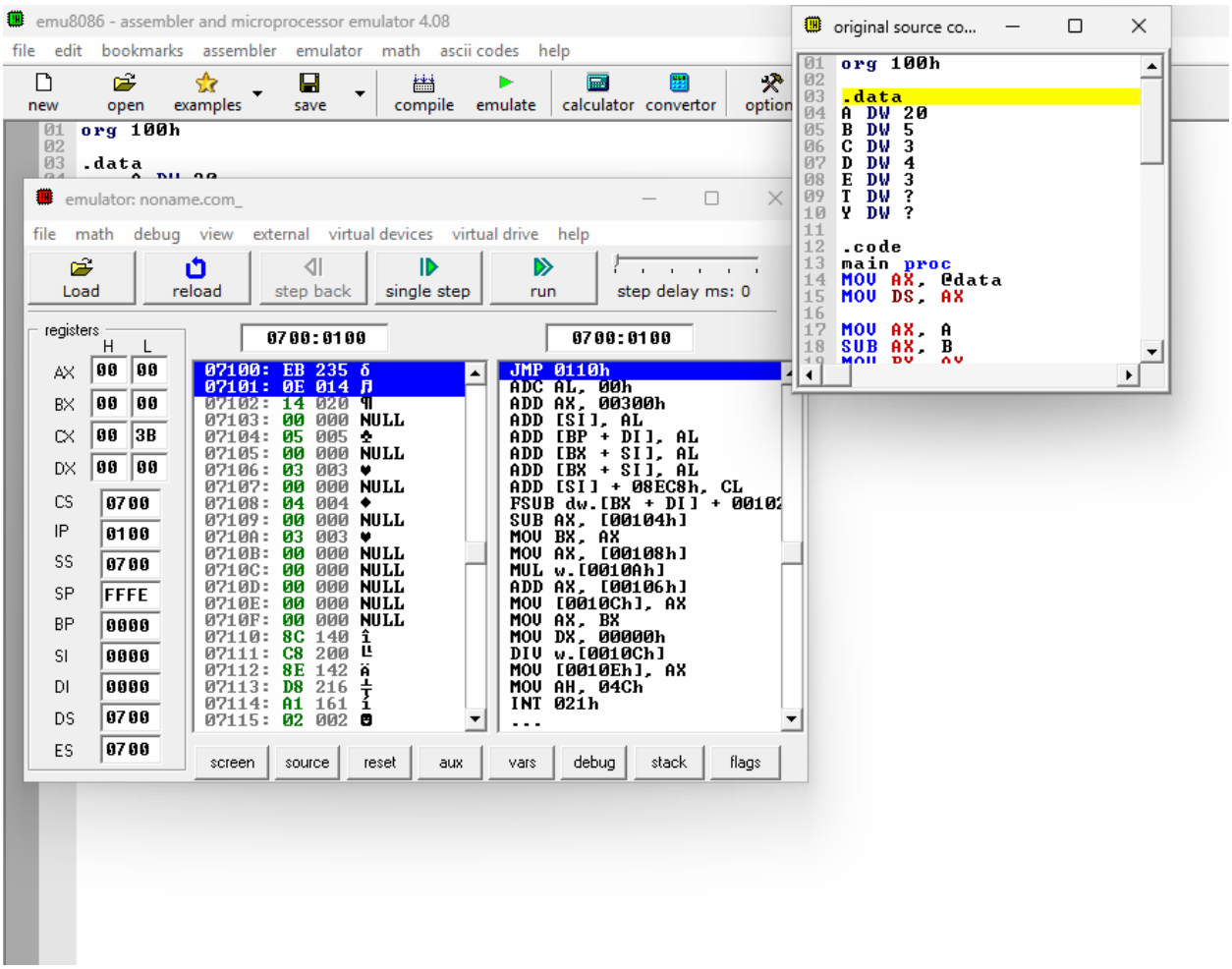
Variables Defined: A = 20, B = 5, C = 3, D = 4, E = 3

Step 1 (Numerator): MOV AX, A -> SUB AX, B -> (20 - 5 = 15). Result saved in BX.

Step 2 (Denominator): MOV AX, D -> MUL E -> (4 * 3 = 12). ADD AX, C -> stored in T.

Step 3 (Division): MOV AX, BX -> MOV DX, 0 -> DIV T -> Y = AX (Quotient).

Step 4 (Exit): MOV AH, 4Ch -> INT 21h -> main endp -> end main.



Assembly Source Code with Output

```
01  org 100h
02
03  .data
04      A DW 20
05      B DW 5
06      C DW 3
07      D DW 4
08      E DW 3
09      T DW ?
10      Y DW ?
11      msg DB 'Result Y = $'
12
13  .code
14  main proc
15      MOV AX, @data
16      MOV DS, AX
17
18      |
19      MOV DX, OFFSET msg
20      MOV AH, 9
21      INT 21h
22
23
24      MOV AX, A
25      SUB AX, B
26      MOV BX, AX
27
28
29      MOV AX, D
30      MUL E
31      ADD AX, C
32      MOV T, AX
33
34
35      MOV AX, BX
36      MOV DX, 0
37      DIV T
38      MOV Y, AX
39
40
41      MOV DX, Y
42      ADD DL, '0'
43      MOV AH, 2
44      INT 21h
45
46
47      MOV AH, 4Ch
48      INT 21h
49  main endp
50
51  end main
```

2. Displaying Text Strings (AH = 9)

String Declaration (msg DB 'Result Y = $'): Defines the bytes for text. The '$' tells INT 21h where to stop printing. Without '$', it prints random memory bytes until it finds 24h (causing 'byte 24h not found' errors).

MOV DX, OFFSET msg: Loads the memory starting address of the string into register DX (DS: DX pair).

MOV AH, 9 & INT 21h: Triggers the DOS video service to render all characters from DS: DX up to '$' onto the screen console.

8086 Assembly: User Input Arithmetic Program

2. Key Concepts & Input Interrupts

- **INT 21h, AH = 1 (Keyboard Input):** Waits for the user to press a key on the keyboard, displays that key on the screen, and automatically stores its ASCII character value in register AL.
- **SUB AL, '0' (ASCII to Number):** When a key like '3' is pressed, AL gets value 33h (48 + 3 = 51). Subtracting '0' (48) converts it back to raw integer value 3 so math can be performed.
- **ADD AL, '0' (Number to ASCII):** Converts the calculated integer back to a printable ASCII code so DOS Service AH=2 can render it on screen.

```
01  .model small
02  .stack 100h
03
04  .data
05      msg1 db 'Enter 1st number: $'
06      msg2 db ' Enter 2nd number: $'
07      msg3 db ' Sum: $'
08
09  .code
10  main proc
11
12      mov ax, @data
13      mov ds, ax
14
15
16      lea dx, msg1
17      mov ah, 9
18      int 21h
19
20
21      mov ah, 1
22      int 21h
23      sub al, 48
24      mov bl, al
25
26
27      lea dx, msg2
28      mov ah, 9
29      int 21h
30
31
32      mov ah, 1
33      int 21h
34      sub al, 48
35      mov bh, al
36
37
38      lea dx, msg3
39      mov ah, 9
40      int 21h
41
42
43      add bl, bh
44      add bl, 48
45
46      ; 7. Display Result
47      mov dl, bl
48      mov ah, 2
49      int 21h
50
51      ; 8. Terminate Program
52      mov ah, 4ch
53      int 21h
54  main endp
55
56  end main
```

Conditional Handling in Assembly Code:

An if-else condition is a control structure used to execute different code blocks based on whether a condition evaluates to **true** or **false**.

- **If (True):** The program executes the first block of code.
- **Else (False):** If the condition is not met, the program skips the if block and executes the else block instead.

How if-else Works in 8086 Assembly (emu8086)

In assembly language, there are no direct if or else keywords.

Conditional Jump Instructions: Jump to a specific label based on the flag status:

1. JE / JZ: Jump if Equal / Jump if Zero
2. JNE / JNZ: Jump if Not Equal
3. JG / JNLE: Jump if Greater (signed)
4. JL / NGE: Jump if Less (signed)
5. JMP: Unconditional jump (used to skip the else block after executing the if block)

High Level Condition	Assembly Level Condition

if (AX == BX) {
}
else {
}
	Assembly Logic:
	CMP AX, BX
	JNE else_label ;
	... (code if AX == BX)
	JMP end_if ;
	else_label:
	... (code if AX != BX)
	end_if:

Jump to the specified location if both values are not equal after comparing them.

```
.MODEL SMALL
.STACK 100h

.DATA
    num1 DB 15
    num2 DB 15

    msg_equal DB 'Numbers are equal!$'
    msg_not_equal DB 'Numbers are NOT equal!$'

.CODE
MAIN PROC

    MOV AX, @DATA
    MOV DS, AX


    MOV AL, num1
    MOV BL, num2


    CMP AL, BL


    JNE ELSE_BLOCK


    LEA DX, msg_equal
    MOV AH, 09h
    INT 21h

    JMP END_IF


ELSE_BLOCK:
    LEA DX, msg_not_equal
    MOV AH, 09h
    INT 21h

END_IF:
    MOV AH, 4Ch
    INT 21h
MAIN ENDP
END MAIN
```

Jump to the specified location if both values equal after comparing them.

```
01  .MODEL SMALL
02  .STACK 100h
03
04  .CODE
05  MAIN PROC
06      MOV AL, 0
07
08      CMP AL, 0
09      JE IS_ZERO
10
11
12      MOV DL, 'N'
13      JMP DISPLAY
14
15  IS_ZERO:
16
17      MOV DL, 'Z'
18
19  DISPLAY:
20      MOV AH, 02h
21      INT 21h
22
23
24      MOV AH, 4Ch
25      INT 21h
26  MAIN ENDP
27  END MAIN
```